

Trilha 7

Matemática Aplicada para Algoritmos, ML, RAG e LLMs

Plano de estudos técnico, progressivo e aplicado para dominar fundamentos matemáticos, algoritmos, machine learning, deep learning, retrieval, RAG, LLMs e leitura de papers em nível avançado/sênior.

Foco	Matemática aplicada à computação moderna e IA em produção.
Nível alvo	Avançado/Sênior técnico.
Entregáveis	Projetos, métricas, papers, arquitetura, checkpoints e critérios de domínio.

Documento formatado a partir do conteúdo anexado pelo usuário.

Sumário

Mapa das seções principais do documento com numeração de páginas gerada automaticamente.

Trilha 7 — Matemática Aplicada para Algoritmos, ML, RAG e LLMs	3
1. Análise inicial da trilha	3
2. Explicação técnica dos tópicos da trilha	4
2.1 Matemática discreta e lógica	4
2.2 Complexidade e algoritmos	5
2.3 Álgebra linear	6
2.4 Probabilidade e estatística	8
2.5 Cálculo e otimização	9
2.6 Matemática para Deep Learning e LLMs	10
2.7 Retrieval e ranking	11
Projeto prático obrigatório	12
2.8 Reinforcement Learning e RLHF	13
2.9 Papers obrigatórios	14
3. Organização como plano de estudos	15
Módulo 1 — Fundamentos matemáticos para computação	15
Módulo 2 — Algoritmos, estruturas de dados e complexidade	16
Módulo 3 — Álgebra linear, estatística e otimização	17
Módulo 4 — Deep Learning, Transformers e LLMs	18
Módulo 5 — Retrieval, RAG, produção, performance e avaliação	19
Módulo 6 — Domínio sênior, papers e liderança técnica	19
4. Critérios de domínio técnico por grande área	20
5. Aplicação prática em sistemas reais	22
6. Profundidade sênior: trade-offs e decisões	23
7. Plano de execução sugerido	24
8. Projetos recomendados para portfólio	25
Projeto 1 — Biblioteca de algoritmos e estruturas de dados	25
Projeto 2 — Sistema de busca semântica	26
Projeto 3 — Avaliador de experimentos A/B	26
Projeto 4 — Mini Transformer educacional	26
Projeto 5 — RAG evaluator completo	26
9. Checklist de domínio técnico	26
10. Indicadores de prontidão para nível avançado/sênior	28

Trilha 7 — Matemática Aplicada para Algoritmos, ML, RAG e LLMs

1. Análise inicial da trilha

Tema central

A trilha é sobre matemática aplicada à computação moderna, com foco em:

- algoritmos e estruturas de dados; entrevistas técnicas; machine learning; deep learning; embeddings; retrieval; RAG; LLMs; leitura e implementação de papers.

Essa trilha não é matemática “acadêmica isolada”. É matemática usada para entender, projetar, avaliar e otimizar sistemas reais baseados em software e inteligência artificial.

Objetivo principal da trilha

O objetivo é formar uma base matemática forte o suficiente para que o estudante consiga:

- provar corretude de algoritmos; analisar complexidade; resolver problemas medium/hard de entrevistas; entender embeddings, vetores, similaridade e ranking; compreender treinamento de modelos; avaliar sistemas de machine learning e RAG; ler papers técnicos sem depender apenas de resumos; tomar decisões arquiteturais envolvendo IA, busca, performance, custo e qualidade.

Em nível sênior, o foco não é apenas “saber calcular”. O profissional precisa entender o impacto técnico das escolhas matemáticas no comportamento do sistema em produção.

Competências técnicas a desenvolver

Ao longo da trilha, o estudante deve desenvolver as seguintes competências:

Competência	Descrição
Raciocínio lógico	Entender proposições, predicados, provas, invariantes e indução.
Análise algorítmica	Avaliar tempo, memória, complexidade, recorrência e escalabilidade.
Modelagem matemática	Representar problemas reais com funções, grafos, matrizes, distribuições e otimização.
Álgebra para IA	Entender vetores, embeddings, similaridade, decomposição, PCA, SVD e tensores.
Estatística aplicada	Avaliar experimentos, métricas, incerteza, testes A/B e performance de modelos.
Otimização	Compreender gradiente, backpropagation, funções de perda e treinamento.
Matemática de LLMs	Entender attention, self-attention, softmax, cross-entropy, perplexity e transformers.
Retrieval e ranking	Projetar e avaliar sistemas de busca, RAG, ANN, BM25, dense retrieval e reranking.
Leitura de papers	Extrair problema, hipótese, matemática central, contribuição e aplicação prática.
Decisão sênior	Justificar trade-offs entre acurácia, latência, custo, complexidade e manutenção.

Classificação por nível

Nível	O que dominar
Básico	Lógica, conjuntos, funções, Big O, vetores, matrizes, probabilidade básica e derivadas.
Intermediário	Grafos, árvores, recorrências, DP, estatística, regressão, gradiente e métricas de ML.
Avançado	SVD, PCA, otimização, backpropagation, transformers, retrieval, ranking e avaliação de RAG.
Sênior	Projetar sistemas matematicamente justificáveis, avaliar trade-offs, ler papers, implementar protótipos e tomar decisões de arquitetura em produção.

2. Explicação técnica dos tópicos da trilha

2.1 Matemática discreta e lógica

O que é

Matemática discreta é a base matemática da computação. Ela trabalha com estruturas contáveis e finitas, como:

- proposições; predicados; conjuntos; relações; funções; indução; combinatória; grafos; árvores; provas; invariantes; recorrências.

Diferente do cálculo, que lida com continuidade, a matemática discreta lida com objetos que aparecem diretamente em programação: listas, árvores, grafos, estados, transições, tabelas, filas, pilhas, mapas e algoritmos.

Para que serve

Serve para compreender formalmente como algoritmos funcionam, por que estão corretos e quais limites possuem.

Exemplos:

- provar que um algoritmo de ordenação sempre termina; provar que uma busca binária retorna o elemento correto; modelar dependências entre serviços como grafos; representar permissões como relações; analisar caminhos em redes; calcular combinações possíveis em problemas de busca; raciocinar sobre estados em sistemas distribuídos.

Por que é importante

Um desenvolvedor sem matemática discreta tende a implementar por tentativa e erro. Um profissional avançado consegue raciocinar sobre:

- corretude; completude; casos extremos; invariantes; crescimento de complexidade; restrições lógicas; propriedades formais do sistema.

Em nível sênior, isso é essencial para revisar algoritmos, detectar bugs sutis, evitar soluções frágeis e justificar escolhas técnicas.

Onde é usado em projetos reais

Conceito	Uso em projeto real
Proposições	Regras de negócio, validações, feature flags, condições booleanas.
Predicados	Filtros, queries, policies, autorização, especificações.
Conjuntos	Deduplicação, permissões, grupos, interseções de dados.
Relações	Modelagem de domínio, relacionamentos, joins, vínculos entre entidades.
Funções	Transformações, mappers, funções puras, pipelines.
Indução	Prova de algoritmos recursivos.
Combinatória	Estimativa de explosão de possibilidades.
Grafos	Redes, dependências, microserviços, recomendações, workflows.
Árvores	DOM, AST, índices, parsers, árvores de decisão.
Invariantes	Garantias que devem permanecer verdadeiras durante execução.
Recorrências	Análise de algoritmos recursivos.

Problemas que resolve

Código com condicionais inconsistentes. Falhas em casos extremos. Loops que não terminam. Recursões mal projetadas. Modelagens ruins de relacionamento. Dificuldade em entender algoritmos. Dificuldade em provar se uma solução está correta. Subestimação de complexidade combinatória.

Erros comuns

Confundir condição necessária com condição suficiente. Não tratar casos de borda. Usar recursão sem entender caso base. Modelar relação como função quando há múltiplas saídas. Ignorar invariantes em loops. Resolver problema de grafo como se fosse simples lista. Achar que “funcionou nos testes” equivale a “está correto”.

Conhecimentos prévios necessários

Lógica booleana. Estruturas básicas de dados. Noções de função. Recursão básica. Leitura de pseudocódigo.

Profundidade esperada para nível sênior

Um profissional sênior deve conseguir:

- provar por indução a corretude de algoritmos recursivos; identificar invariantes em loops e algoritmos de busca; modelar problemas reais com grafos, árvores e relações; estimar explosão combinatória antes da implementação; revisar soluções algorítmicas de outros desenvolvedores; decidir quando formalismo é necessário e quando é excesso.

2.2 Complexidade e algoritmos

O que é

Complexidade mede o custo computacional de um algoritmo em função do tamanho da entrada.

A trilha inclui:

- Big O; Big Ω; Big Θ; recursão; Master Theorem; sorting; searching; hashing; trees; graphs; dynamic programming; greedy; backtracking; sliding window; two pointers; monotonic stack; union-find; shortest path; MST.

Para que serve

Serve para responder perguntas como:

- esse algoritmo escala? quanto tempo vai levar com 1 milhão de registros? quanto de memória será consumido? qual estrutura de dados reduz custo? qual abordagem é viável em produção? qual solução passa em entrevista técnica?

Por que é importante

A diferença entre $O(n)$, $O(n \log n)$ e $O(n^2)$ pode ser irrelevante com 100 registros, mas crítica com milhões de eventos, documentos ou embeddings.

Em sistemas reais, complexidade afeta:

- latência; custo de cloud; throughput; consumo de memória; escalabilidade; experiência do usuário; estabilidade em produção.

Aplicação em projetos reais

Tópico	Aplicação real
Big O	Avaliar loops, queries, pipelines e processamento em lote.
Recursão	Parsers, árvores, grafos, backtracking e divide and conquer.
Sorting	Ordenação de rankings, relatórios, logs e resultados de busca.

Tópico	Aplicação real
Searching	Busca em listas, índices, árvores, caches e bancos.
Hashing	Maps, caches, deduplicação, particionamento, sharding.
Trees	Índices, AST, hierarquias, menus, estruturas organizacionais.
Graphs	Rotas, dependências, recomendações, workflows, redes sociais.
DP	Otimização, sequências, problemas de custo mínimo.
Greedy	Escalonamento, seleção local, heurísticas.
Backtracking	Constraint solving, combinações, alocação.
Sliding window	Processamento de streams, séries temporais, strings.
Union-find	Componentes conectados, redes, agrupamentos.
Shortest path	Rotas, grafos de dependência, custo mínimo.
MST	Redes, conectividade, otimização de infraestrutura.

Problemas que resolve

APIs lentas por algoritmos quadráticos. Jobs batch demorando horas por escolha errada de estrutura. Uso excessivo de memória. Busca ineficiente em grandes coleções. Soluções de entrevista que não passam nos limites. Sistemas que funcionam localmente, mas falham em escala.

Erros comuns

Medir performance apenas com poucos dados. Usar lista onde deveria usar hash map. Ignorar complexidade de operações internas. Confundir complexidade média com pior caso. Aplicar DP sem entender estado e transição. Usar greedy em problema que exige ótimo global. Resolver grafo sem definir vértices, arestas e pesos.

Profundidade sênior

O critério sênior não é apenas resolver problemas medium/hard. É conseguir:

- identificar o padrão algorítmico correto; justificar complexidade temporal e espacial; comparar duas soluções viáveis; prever impacto em produção; transformar algoritmo em código limpo e testável; avaliar trade-off entre exatidão, heurística, custo e latência.

Exemplo: em um sistema de recomendação, uma busca exata pode ser matematicamente melhor, mas inviável em latência. Um sênior pode escolher busca aproximada com recall controlado, monitoramento e fallback.

2.3 Álgebra linear

O que é

Álgebra linear é a matemática dos vetores, matrizes, espaços e transformações lineares.

A trilha inclui:

- vetores; matrizes; produto escalar; norma; distância; cosine similarity; espaços vetoriais; base; dimensão; autovalores; autovetores; SVD; PCA; tensor operations; tensor decomposition; CP decomposition; Tucker decomposition.

Para que serve

Serve para representar dados como objetos matemáticos manipuláveis.

Em IA moderna, praticamente tudo vira vetor:

- palavras; documentos; imagens; usuários; produtos; queries; respostas; comportamentos; transações.

Embeddings são representações vetoriais. Retrieval usa similaridade vetorial. Redes neurais operam com matrizes e tensores.

Por que é importante

Sem álgebra linear, o estudante até consegue usar bibliotecas, mas não entende profundamente:

- por que embeddings funcionam; como medir similaridade; como reduzir dimensionalidade; como modelos transformam dados; por que matrizes grandes custam caro; como operações tensoriais afetam memória e GPU.

Aplicação em projetos reais

Conceito	Aplicação
Vetores	Embeddings de texto, imagem, produto e usuário.
Matrizes	Pesos de redes neurais, batch processing, transformações.
Produto escalar	Similaridade, attention score, projeções.
Norma	Magnitude de vetores, regularização, normalização.
Distância	Clusterização, busca vetorial, detecção de anomalias.
Cosine similarity	Retrieval semântico, recomendação, comparação de documentos.
Espaço vetorial	Representação semântica de dados.
Base e dimensão	Compressão, representação mínima, feature space.
Autovalores/autovetores	PCA, estabilidade, análise espectral.
SVD	Compressão, recomendação, redução de ruído.
PCA	Redução de dimensionalidade e visualização.
Tensores	Deep learning, imagens, batches, sequências.
CP/Tucker	Compressão e fatoração de tensores multidimensionais.

Problemas que resolve

Busca semântica ineficiente. Embeddings mal avaliados. Alta dimensionalidade sem controle. Modelos caros demais. Features redundantes. Sistemas de recomendação pouco escaláveis. Falta de entendimento sobre similaridade.

Erros comuns

Usar cosine similarity sem normalizar corretamente. Confundir distância euclidiana com similaridade semântica. Achar que mais dimensão sempre é melhor. Usar PCA sem padronizar dados. Ignorar custo de multiplicação matricial. Não entender diferença entre matriz, vetor e tensor. Tratar embedding como “mágica” e não como representação aproximada.

Profundidade sênior

Um profissional sênior precisa decidir:

- qual métrica de similaridade usar; quando reduzir dimensionalidade; como balancear qualidade versus custo; quando usar embedding denso, sparse ou híbrido; como avaliar degradação após compressão; quando uma decomposição tensorial vale a complexidade; como explicar limitações de embeddings para produto, negócio e engenharia.

2.4 Probabilidade e estatística

O que é

Probabilidade modela incerteza. Estatística extrai conclusões a partir de dados.

A trilha inclui:

- variáveis aleatórias; distribuições; esperança; variância; covariância; correlação; Bayes; intervalos de confiança; testes de hipótese; p-value; A/B testing; regressão; bootstrap; MCMC; inferência variacional; estatística bayesiana.

Para que serve

Serve para avaliar modelos, experimentos, produtos e decisões sob incerteza.

Em sistemas reais, raramente temos certeza absoluta. Trabalhamos com:

- amostras; ruído; tendências; inferência; risco; erro; confiança; variabilidade.

Por que é importante

Sem estatística, o profissional pode tomar decisões erradas com base em métricas enganosas.

Exemplo: um modelo parece ter melhorado de 82% para 84% de acurácia. Isso é melhoria real ou ruído amostral? Depende de tamanho da amostra, variância, intervalo de confiança e teste estatístico.

Aplicação em projetos reais

Conceito	Aplicação
Variáveis aleatórias	Modelar eventos incertos: cliques, falhas, conversões.
Distribuições	Modelar tempos de resposta, erros, demanda, comportamento.
Esperança	Valor médio esperado de uma métrica.
Variância	Instabilidade, dispersão, risco.
Covariância/correlação	Relação entre features e métricas.
Bayes	Atualização de crenças, classificação, risco.
Intervalos de confiança	Incerteza sobre métricas.
Testes de hipótese	Validar se diferença é estatisticamente relevante.
P-value	Medir evidência contra hipótese nula.
A/B testing	Avaliação de produto, ranking, UX, modelos.
Regressão	Previsão, causalidade aproximada, modelagem.
Bootstrap	Estimar incerteza sem assumir distribuição forte.
MCMC	Amostragem em modelos probabilísticos complexos.
Inferência variacional	Aproximação eficiente de distribuições difíceis.
Estatística bayesiana	Decisão probabilística com prior e posterior.

Problemas que resolve

Decisões baseadas em amostras pequenas. A/B tests mal interpretados. Métricas de ML avaliadas sem confiança estatística. Falta de noção de incerteza. Confusão entre correlação e causalidade. Modelos aparentemente bons, mas instáveis. Produtos alterados com base em falso positivo.

Erros comuns

Usar p-value como “probabilidade da hipótese ser verdadeira”. Ignorar tamanho da amostra. Parar A/B test cedo demais. Confundir significância estatística com relevância prática. Usar média sem olhar distribuição. Ignorar outliers. Comparar modelos sem intervalo de confiança. Confundir correlação com causa.

Profundidade sênior

Um sênior deve conseguir:

- desenhar experimentos confiáveis; interpretar métricas com incerteza; avaliar viés de amostragem; decidir quando um resultado é acionável; explicar limites estatísticos para stakeholders; escolher métrica alinhada ao objetivo de negócio; revisar dashboards e evitar conclusões falsas.

2.5 Cálculo e otimização

O que é

Cálculo estuda variações contínuas. Otimização busca encontrar valores que minimizam ou maximizam uma função.

A trilha inclui:

- funções; limites; derivadas; gradiente; Jacobiano; Hessiana; regra da cadeia; otimização; gradient descent; Adam; convexidade; KKT; dualidade; regularização.

Para que serve

Serve para entender como modelos aprendem.

Em machine learning, normalmente definimos uma função de perda e ajustamos parâmetros para minimizar essa perda.

Exemplo:

modelo -> previsão -> erro -> loss -> gradiente -> atualização dos pesos

Por que é importante

Sem cálculo e otimização, backpropagation, redes neurais, SVM, regressão e fine-tuning viram apenas chamadas de biblioteca.

O estudante precisa entender:

- o que significa minimizar uma loss; por que derivada indica direção de mudança; por que gradiente aponta crescimento; por que descida do gradiente reduz erro; por que learning rate pode explodir ou travar treinamento; por que regularização reduz overfitting.

Aplicação em projetos reais

Conceito	Aplicação
Funções	Modelos, losses, transformações.
Derivadas	Taxa de variação, sensibilidade.
Gradiente	Atualização de parâmetros.
Jacobiano	Derivadas de funções vetoriais.
Hessiana	Curvatura, segunda ordem, análise de otimização.
Regra da cadeia	Backpropagation.
Gradient descent	Treinamento básico de modelos.
Adam	Otimizador adaptativo usado em deep learning.

Conceito	Aplicação
Convexidade	Garantias de ótimo global em alguns problemas.
KKT	Otimização com restrições.
Dualidade	SVM, otimização convexa, problemas restritos.
Regularização	Controle de overfitting e estabilidade.

Problemas que resolve

Treinamento instável. Overfitting. Underfitting. Escolha inadequada de learning rate. Falta de entendimento de loss. Modelos sensíveis demais. Má interpretação de convergência.

Erros comuns

Achar que otimização sempre encontra ótimo global. Usar Adam sem entender hiperparâmetros. Ignorar escala das features. Não diferenciar loss de métrica de negócio. Confundir gradiente zero com bom modelo. Aplicar regularização sem avaliar impacto. Não entender exploding/vanishing gradients.

Profundidade sênior

Um sênior deve conseguir:

- diagnosticar problemas de treinamento; escolher loss adequada ao problema; ajustar regularização; avaliar estabilidade; entender custo computacional do treinamento; explicar trade-off entre qualidade, custo e tempo; decidir quando treinar, fine-tunar, usar embedding pronto ou usar API externa.

2.6 Matemática para Deep Learning e LLMs

O que é

É o conjunto de conceitos matemáticos necessários para entender redes neurais modernas, principalmente Transformers e LLMs.

A trilha inclui:

- softmax; cross-entropy; backpropagation; attention; self-attention; positional encoding; LayerNorm; embeddings; Transformers; KL divergence; perplexity; contrastive learning; quantization; information theory.

O paper Attention Is All You Need introduziu a arquitetura Transformer baseada em mecanismos de atenção, removendo a dependência central de recorrência e convolução nos modelos de sequência originais.

Para que serve

Serve para entender como modelos modernos de linguagem funcionam matematicamente.

Em vez de tratar LLM como caixa-preta, o estudante aprende:

- como tokens viram vetores; como attention calcula relevância entre tokens; como softmax transforma scores em distribuição; como cross-entropy mede erro; como backpropagation ajusta pesos; como quantization reduz custo; como perplexity avalia capacidade de previsão.

Por que é importante

Para trabalhar com IA de forma técnica, não basta saber usar prompts ou APIs. É necessário entender:

- limites do modelo; custo de inferência; impacto de contexto; compressão; fine-tuning; avaliação; alinhamento; arquitetura de RAG; riscos de hallucination; métricas de qualidade.

Aplicação em projetos reais

Conceito	Aplicação
Softmax	Classificação, attention weights, distribuição de probabilidade.
Cross-entropy	Loss para classificação e modelagem de linguagem.
Backpropagation	Treinamento e fine-tuning.
Attention	Relações entre tokens ou partes de entrada.
Self-attention	Base dos Transformers.
Positional encoding	Informação de ordem dos tokens.
LayerNorm	Estabilização do treinamento.
Embeddings	Representação semântica.
Transformers	LLMs, BERT, GPT, modelos multimodais.
KL divergence	Comparação entre distribuições.
Perplexity	Avaliação de linguagem.
Contrastive learning	Aprender representações por aproximação/afastamento.
Quantization	Redução de memória e custo.
Information theory	Entropia, compressão, incerteza.

Problemas que resolve

Uso superficial de LLMs. Falta de entendimento sobre hallucination. RAG mal avaliado. Fine-tuning sem critério. Escolha errada entre prompt engineering, RAG e treinamento. Uso caro demais de modelos. Baixa qualidade em retrieval ou geração.

Erros comuns

Achar que LLM “entende” como humano. Usar perplexity como única métrica de qualidade. Aplicar fine-tuning quando RAG resolveria. Usar RAG quando prompt simples resolveria. Ignorar janela de contexto. Não medir custo por token. Não avaliar bias, segurança e confiabilidade. Confundir embedding model com generative model.

Profundidade sênior

Um sênior deve decidir:

- quando usar modelo fechado ou aberto; quando usar RAG, fine-tuning, LoRA ou prompt engineering; como balancear custo, latência e qualidade; como avaliar hallucination; como versionar prompts, datasets e modelos; como definir guardrails; como monitorar qualidade em produção; como explicar limitações técnicas para negócio.

2.7 Retrieval e ranking

O que é

Retrieval é o processo de buscar documentos, trechos ou entidades relevantes para uma consulta. Ranking é o processo de ordenar os resultados por relevância.

A trilha inclui:

- TF-IDF; BM25; cosine similarity; dense retrieval; hybrid search; MRR; MAP; NDCG; Recall@K; Precision@K; reranking; ANN; HNSW; IVF.

RAG combina geração de linguagem com recuperação de conhecimento externo; o paper original de Retrieval-Augmented Generation descreve a combinação de memória paramétrica do modelo com memória não paramétrica baseada em índice vetorial.

Para que serve

Serve para construir sistemas como:

- busca interna; search de e-commerce; recomendação; busca semântica; assistentes com documentos; RAG corporativo; suporte automatizado; análise de contratos; chatbot com base de conhecimento.

Por que é importante

Em RAG, a resposta final depende diretamente da qualidade do retrieval. Se o sistema recupera contexto ruim, o LLM tende a responder mal, mesmo que o modelo seja excelente.

A qualidade de um sistema RAG depende de:

- chunking; embeddings; índice; ranking; reranking; filtros; metadados; avaliação; observabilidade; feedback loop.

Aplicação em projetos reais

Conceito	Aplicação
TF-IDF	Busca lexical simples e baseline.
BM25	Search textual robusto em documentos.
Cosine similarity	Similaridade entre embeddings.
Dense retrieval	Busca semântica vetorial.
Hybrid search	Combina lexical e vetorial.
MRR	Avalia posição do primeiro resultado relevante.
MAP	Avalia precisão média em múltiplos resultados.
NDCG	Avalia ranking considerando relevância graduada.
Recall@K	Mede se o relevante apareceu no top K.
Precision@K	Mede qualidade dos K primeiros resultados.
Reranking	Reordena candidatos com modelo mais caro e preciso.
ANN	Busca aproximada em escala.
HNSW	Índice vetorial baseado em grafos.
IVF	Índice por agrupamento/particionamento vetorial.

HNSW é uma técnica de busca aproximada de vizinhos mais próximos baseada em grafos hierárquicos, relevante para reduzir latência em busca vetorial de alta dimensionalidade.

Projeto prático obrigatório

Implementar métricas de retrieval para avaliar RAG

O projeto deve conter:

- base de documentos; conjunto de perguntas; respostas esperadas; documentos relevantes esperados; pipeline de chunking; embeddings; busca lexical; busca vetorial; busca híbrida; reranking; cálculo de:
- Recall@K; Precision@K; MRR; MAP; NDCG; relatório comparando abordagens.

Erros comuns

Avaliar RAG apenas olhando a resposta final. Não medir retrieval separadamente. Usar chunk muito grande ou muito pequeno sem teste. Não considerar metadados. Não ter ground truth. Não versionar embeddings. Trocar modelo de embedding sem reavaliar índice. Usar busca vetorial para tudo e ignorar BM25. Não medir latência por etapa.

Profundidade sênior

Um sênior deve conseguir:

- desenhar benchmark de retrieval; escolher métrica alinhada ao caso de uso; separar erro de retrieval de erro de geração; decidir entre BM25, dense, hybrid e reranking; avaliar custo de índice vetorial; projetar estratégia de atualização dos documentos; definir fallback quando retrieval falha; monitorar drift da base e qualidade das respostas.

2.8 Reinforcement Learning e RLHF

O que é

Reinforcement Learning é aprendizado por interação com ambiente. Um agente executa ações, recebe recompensas e aprende uma política para maximizar retorno esperado.

A trilha inclui:

- MDP; policy; value function; Q-learning; policy gradient; PPO; reward modeling; preference learning; RLHF; alignment.

PPO é uma família de métodos de policy gradient que alterna coleta de dados por interação com o ambiente e otimização de um objetivo substituto, sendo uma base importante para discussões modernas de RLHF.

Para que serve

Serve para entender:

- agentes; sistemas de decisão; otimização por recompensa; alinhamento de modelos; feedback humano; preferência; comportamento emergente; trade-offs entre exploração e exploração do que já funciona.

Por que é importante

Em LLMs, RLHF aparece como uma estratégia para alinhar comportamento do modelo com preferências humanas. Isso ajuda a entender por que modelos seguem instruções, evitam certos comportamentos e priorizam respostas mais úteis.

Aplicação em projetos reais

Conceito	Aplicação
MDP	Modelagem de estados, ações, transições e recompensas.
Policy	Estratégia de decisão de agente.
Value function	Estimativa de valor futuro.
Q-learning	Aprendizado de valor ação-estado.
Policy gradient	Otimização direta da política.
PPO	Treinamento mais estável em políticas.
Reward modeling	Modelo que estima preferência/recompensa.
Preference learning	Aprender a partir de comparações humanas.
RLHF	Alinhamento de LLMs com feedback humano.

Conceito	Aplicação
Alignment	Redução de desalinhamento entre objetivo e comportamento.

Problemas que resolve

Agentes sem estratégia clara. Modelos que otimizam métrica errada. Sistemas que maximizam recompensa de forma indesejada. Falta de alinhamento entre comportamento do modelo e intenção humana. Decisões sequenciais difíceis de modelar com supervised learning simples.

Erros comuns

Definir reward mal especificada. Confundir reward alta com comportamento desejável. Ignorar reward hacking. Aplicar RL quando supervised learning resolveria. Não controlar exploração. Avaliar agente somente em ambiente simples. Não considerar segurança e limites de ação.

Profundidade sênior

Um sênior deve conseguir:

- avaliar se RL é necessário; projetar reward com cuidado; identificar risco de comportamento indesejado; separar política, valor e ambiente; entender limites de RLHF; avaliar custo de coleta de preferência humana; definir mecanismos de segurança para agentes.

2.9 Papers obrigatórios

Por que estudar papers

Papers são a principal forma de acompanhar avanços técnicos em IA. O objetivo não é decorar fórmulas, mas aprender a extrair:

- problema atacado; hipótese central; solução proposta; matemática usada; arquitetura; experimentos; limitações; aplicação prática; possibilidade de implementação mínima.

Papers da trilha

Paper	O que estudar
Attention Is All You Need	Transformer, self-attention, positional encoding, encoder-decoder.
BERT	Pré-treinamento bidirecional, masked language modeling, fine-tuning.
GPT-3	Scaling, few-shot learning, in-context learning.
RAG	Combinação de geração com retrieval externo.
ReAct	Intercalação de raciocínio e ação em LLMs.
LoRA	Fine-tuning eficiente com matrizes de baixo rank.
QLoRA	Fine-tuning eficiente com quantização em 4 bits.
ColBERT	Retrieval com late interaction contextualizada.
DDPM	Modelos de difusão e denoising probabilístico.

BERT propôs pré-treinamento bidirecional profundo usando contexto à esquerda e à direita, com fine-tuning simples para várias tarefas de NLP. GPT-3 demonstrou forte desempenho few-shot com um modelo autoregressivo de 175 bilhões de parâmetros, usando tarefas especificadas via texto sem atualização de gradiente no momento da inferência.

LoRA reduz custo de adaptação congelando pesos do modelo base e adicionando matrizes treináveis de baixo rank; QLoRA combina LoRA com modelo quantizado em 4 bits para reduzir uso de memória no fine-tuning. ReAct propõe intercalar raciocínio e ações para permitir que LLMs consultem ambientes externos durante a resolução de tarefas. ColBERT usa late interaction para manter expressividade de

modelos contextualizados com custo de busca menor. DDPM é uma base importante para modelos de difusão, com geração baseada em processo probabilístico de remoção de ruído.

Template de resumo técnico de paper

Para cada paper, produzir:

1. Problema

Qual limitação técnica o paper tenta resolver?

2. Solução

Qual é a ideia central proposta?

3. Matemática central

Quais equações, funções, distribuições, losses ou métricas sustentam a solução?

4. Arquitetura

Como os componentes se conectam?

5. Experimentos

Como o paper prova que a solução funciona?

6. Limitações

O que o paper não resolve?

7. Aplicação prática

Onde essa ideia pode ser usada em um sistema real?

8. Implementação mínima

Qual protótipo simples pode ser feito para validar a ideia?

3. Organização como plano de estudos

Módulo 1 — Fundamentos matemáticos para computação

Objetivo

Construir a base lógica e discreta necessária para algoritmos, provas, estruturas de dados e modelagem computacional.

Conteúdos

Proposições. Predicados. Conjuntos. Relações. Funções. Indução. Provas. Invariantes. Combinatória. Recorrências. Grafos. Árvores.

Ordem recomendada

Lógica proposicional. Predicados. Conjuntos. Relações e funções. Técnicas de prova. Indução. Invariantes. Combinatória. Recorrências. Grafos e árvores.

Conceitos obrigatórios

Implicação. Contrapositiva. Quantificadores. Pertencimento. Subconjunto. Produto cartesiano. Função injetora, sobrejetora e bijetora. Prova por indução. Invariante de loop. Relação de recorrência. Grau, caminho, ciclo e conectividade em grafos.

Práticas recomendadas

Escrever provas curtas. Modelar problemas reais com conjuntos e relações. Identificar invariantes em algoritmos simples. Transformar regras de negócio em predicados. Representar dependências com grafos.

Exercícios técnicos

Provar corretude da busca binária. Provar que um algoritmo recursivo termina. Modelar permissões de usuário com conjuntos. Modelar dependência entre microsserviços como grafo. Resolver problemas de contagem simples.

Mini projeto

Validador lógico de regras de negócio

Criar um pequeno motor que avalia regras como:

cliente ativo AND limite disponível AND não bloqueado

O projeto deve conter:

- parser simples; operadores lógicos; avaliação booleana; testes unitários; casos de borda.

Critérios de conclusão

Explicar diferença entre proposição e predicado. Provar algoritmo simples por indução. Identificar invariante de loop. Modelar problema real como grafo. Resolver recorrências básicas.

Nível esperado ao finalizar

Básico forte para intermediário.

Módulo 2 — Algoritmos, estruturas de dados e complexidade

Objetivo

Desenvolver capacidade de resolver problemas computacionais com análise formal de custo.

Conteúdos

Big O, Big Ω e Big Θ . Recursão. Master Theorem. Sorting. Searching. Hashing. Trees. Graphs. Dynamic programming. Greedy. Backtracking. Sliding window. Two pointers. Monotonic stack. Union-find. Shortest path. MST.

Ordem recomendada

Big O/ Ω / Θ . Arrays, strings, hash maps e sets. Sorting/searching. Recursão e Master Theorem. Trees. Graphs. BFS/DFS. Sliding window/two pointers. Stack/monotonic stack. Greedy. Backtracking. Dynamic programming. Union-find. Shortest path. MST.

Conceitos obrigatórios

Complexidade temporal. Complexidade espacial. Pior caso, caso médio e melhor caso. Recorrências. BFS e DFS. Estado e transição em DP. Propriedade gulosa. Relaxamento de arestas. Compressão de caminho em union-find.

Práticas recomendadas

Resolver problemas por padrão. Sempre escrever complexidade. Fazer dry run. Testar casos extremos. Comparar solução bruta e solução otimizada. Refatorar código de entrevista para código limpo.

Exercícios técnicos

30 problemas fáceis. 50 problemas médios. 20 problemas difíceis. Implementar do zero:

- binary search; merge sort; quick sort; heap; trie; BFS; DFS; Dijkstra; union-find; knapsack; longest increasing subsequence.

Mini projeto

Engine de recomendação baseada em grafos

Criar um sistema que recebe usuários, produtos e interações. Representar como grafo e gerar recomendações simples por vizinhança.

Deve conter:

- modelagem de grafo; BFS; ranking simples; análise de complexidade; testes; relatório de trade-offs.

Critérios de conclusão

Resolver problemas medium com autonomia. Explicar complexidade sem decorar. Identificar padrão algorítmico adequado. Implementar grafos e árvores do zero. Resolver problemas hard com orientação mínima.

Nível esperado ao finalizar

Intermediário forte para avançado.

Módulo 3 — Álgebra linear, estatística e otimização

Objetivo

Construir a base matemática para machine learning, embeddings, deep learning e avaliação de modelos.

Conteúdos

Vetores. Matrizes. Produto escalar. Normas. Distâncias. Cosine similarity. Espaços vetoriais. Base e dimensão. Autovalores/autovetores. SVD. PCA. Probabilidade. Distribuições. Esperança e variância. Covariância e correlação. Bayes. Testes de hipótese. A/B testing. Regressão. Derivadas. Gradiente. Otimização. Gradient descent. Adam. Regularização.

Ordem recomendada

Vetores e matrizes. Produto escalar, norma e distância. Espaços vetoriais. SVD e PCA. Probabilidade básica. Estatística descritiva. Inferência estatística. Testes de hipótese. Regressão. Derivadas e gradiente. Otimização. Regularização.

Conceitos obrigatórios

Similaridade vetorial. Redução de dimensionalidade. Distribuição de probabilidade. Valor esperado. Variância. Teste estatístico. Gradiente. Loss function. Overfitting. Regularização.

Práticas recomendadas

Implementar operações vetoriais sem biblioteca inicialmente. Comparar métricas de distância. Criar simulações estatísticas. Rodar A/B test sintético. Implementar regressão linear do zero. Implementar gradient descent do zero.

Exercícios técnicos

Calcular cosine similarity entre documentos. Implementar PCA simples. Simular distribuição binomial e normal. Calcular intervalo de confiança. Fazer teste de hipótese. Implementar regressão linear. Comparar gradient descent com solução fechada.

Mini projeto

Sistema de análise estatística de experimentos

Criar uma aplicação que recebe resultados de experimento A/B e calcula:

- taxa de conversão; diferença absoluta; diferença relativa; intervalo de confiança; teste de hipótese; recomendação técnica.

Critérios de conclusão

Explicar embeddings como vetores. Comparar similaridade e distância. Avaliar experimento com incerteza. Implementar regressão simples. Explicar gradiente e regularização.

Nível esperado ao finalizar

Avançado inicial.

Módulo 4 — Deep Learning, Transformers e LLMs

Objetivo

Entender matematicamente como redes neurais modernas funcionam, com foco em Transformers e modelos de linguagem.

Conteúdos

Softmax. Cross-entropy. Backpropagation. Embeddings. Attention. Self-attention. Positional encoding. LayerNorm. Transformers. KL divergence. Perplexity. Contrastive learning. Quantization. Information theory.

Ordem recomendada

Perceptron e MLP. Loss functions. Backpropagation. Softmax e cross-entropy. Embeddings. Attention. Self-attention. Transformer encoder. Transformer decoder. Métricas de linguagem. Quantization. Contrastive learning.

Conceitos obrigatórios

Tokenização. Embedding. Query, Key e Value. Attention score. Multi-head attention. Feed-forward network. Residual connection. Normalização. Perplexity. Divergência entre distribuições. Quantização.

Práticas recomendadas

Implementar softmax manualmente. Implementar cross-entropy. Implementar attention simplificada. Visualizar matriz de attention. Criar mini Transformer educacional. Comparar inferência com diferentes tamanhos de contexto.

Exercícios técnicos

Implementar softmax numericamente estável. Implementar cross-entropy. Implementar self-attention com matrizes pequenas. Calcular perplexity em exemplos simples. Comparar embeddings por similaridade. Simular quantização simples.

Mini projeto

Mini Transformer educacional

Criar uma implementação simplificada com:

- embeddings; positional encoding; self-attention; feed-forward; forward pass; comentários matemáticos; testes para shapes; explicação de cada matriz.

Critérios de conclusão

Explicar attention sem analogias superficiais. Derivar softmax e cross-entropy em alto nível. Entender fluxo do backpropagation. Explicar como um Transformer processa tokens. Relacionar custo computacional com tamanho de contexto.

Nível esperado ao finalizar

Avançado.

Módulo 5 — Retrieval, RAG, produção, performance e avaliação

Objetivo

Aplicar matemática em sistemas reais de busca, RAG e avaliação de qualidade em produção.

Conteúdos

TF-IDF. BM25. Cosine similarity. Dense retrieval. Hybrid search. Reranking. ANN. HNSW. IVF. MRR. MAP. NDCG. Recall@K. Precision@K. Observabilidade de RAG. Segurança em RAG. Avaliação offline e online. Latência e custo.

Ordem recomendada

Busca lexical. TF-IDF. BM25. Embeddings. Busca vetorial. Métricas de retrieval. Busca híbrida. Reranking. ANN. HNSW/IVF. Avaliação de RAG. Observabilidade. Segurança. Otimização de custo.

Conceitos obrigatórios

Chunking. Indexação. Similaridade lexical. Similaridade semântica. Recall versus precision. Ground truth. Reranking. Latência P95/P99. Custo por query. Segurança de contexto. Prompt injection em RAG. Data freshness.

Práticas recomendadas

Avaliar retrieval antes da resposta final. Criar dataset de perguntas e documentos relevantes. Medir top K. Comparar lexical, vetorial e híbrido. Usar logs para análise de falha. Versionar embeddings. Monitorar drift. Separar pipeline offline e online.

Exercícios técnicos

Implementar TF-IDF. Implementar BM25 simplificado. Calcular Recall@K e Precision@K. Implementar MRR. Implementar NDCG. Comparar dense retrieval com BM25. Testar reranking. Avaliar impacto do tamanho do chunk.

Mini projeto

RAG evaluator completo

Criar um sistema com:

- ingestion de documentos; chunking; embeddings; índice vetorial; busca lexical; busca híbrida; reranking; geração com LLM; avaliação de retrieval; avaliação da resposta; dashboard simples com métricas.

Critérios de conclusão

Separar problema de retrieval de problema de geração. Justificar métrica escolhida. Medir latência e custo. Identificar falhas de chunking. Definir estratégia de atualização do índice. Documentar trade-offs.

Nível esperado ao finalizar

Avançado para sênior.

Módulo 6 — Domínio sênior, papers e liderança técnica

Objetivo

Transformar conhecimento matemático em decisão técnica, arquitetura, revisão, documentação e liderança.

Conteúdos

Leitura de papers. Attention Is All You Need. BERT. GPT-3. RAG. ReAct. LoRA. QLoRA. CoBERT. DDPM. RL. RLHF. PPO. Reward modeling. Preference learning. Alignment. Decisão arquitetural. RFC técnico. ADR. Benchmark. Avaliação de risco. Custo em produção.

Ordem recomendada

Template de leitura de paper. Transformers. BERT e GPT-3. RAG e ColBERT. LoRA e QLoRA. ReAct. DDPM. RL, PPO e RLHF. Projeto final. Documento arquitetural. Apresentação técnica.

Conceitos obrigatórios

Paper reading. Formulação de problema. Hipótese. Baseline. Métrica. Ablation. Limitações. Implementação mínima. Benchmark. Decisão técnica justificada. Custo técnico. Risco operacional. Manutenibilidade.

Práticas recomendadas

Ler paper com foco em implementação. Fazer resumo técnico. Reproduzir parte pequena do paper. Comparar com baseline. Criar ADR para cada decisão importante. Apresentar trade-offs. Fazer code review técnico. Criar benchmark reproduzível.

Exercícios técnicos

Resumir cada paper obrigatório. Implementar mini attention. Implementar métrica de retrieval. Implementar LoRA conceitual em camada linear pequena. Criar benchmark BM25 versus dense retrieval. Criar RFC de arquitetura para RAG corporativo. Fazer análise de risco de uso de LLM em produção.

Mini projeto final

Plataforma RAG com avaliação, observabilidade e governança

Deve conter:

- API; banco de dados; armazenamento de documentos; pipeline de ingestão; chunking; embedding; índice vetorial; busca híbrida; reranking; geração; avaliação offline; logs; métricas; feedback do usuário; versionamento de prompt; versionamento de embedding; testes; documentação; ADRs; análise de custo; análise de segurança.

Critérios de conclusão

Ler e explicar papers com autonomia. Implementar protótipos mínimos. Justificar decisões arquiteturais. Avaliar custo, risco e performance. Produzir documentação técnica de nível profissional. Conduzir revisão técnica de solução de IA.

Nível esperado ao finalizar

Sênior técnico em matemática aplicada para algoritmos, ML, retrieval e LLMs.

4. Critérios de domínio técnico por grande área

Matemática discreta e lógica

Nível	Critério
Iniciante	Entende proposições, conjuntos e funções básicas.
Intermediário	Aplica indução, relações, grafos e árvores em problemas simples.
Avançado	Prova corretude de algoritmos e usa invariantes.
Sênior	Modela problemas reais formalmente, revisa soluções e identifica falhas lógicas antes da produção.

Algoritmos e complexidade

Nível	Critério
Iniciante	Reconhece Big O e implementa estruturas básicas.

Nível	Critério
Intermediário	Resolve problemas com hash, árvore, grafo, recursão e sorting.
Avançado	Resolve DP, greedy, shortest path, union-find e problemas medium/hard.
Sênior	Escolhe algoritmo considerando escala, latência, memória, manutenção e custo.

Álgebra linear

Nível	Critério
Iniciante	Entende vetores, matrizes, produto escalar e distância.
Intermediário	Usa cosine similarity, PCA e operações matriciais.
Avançado	Entende SVD, autovalores, tensores e embeddings.
Sênior	Decide representações vetoriais, métricas e compressões com base em qualidade, custo e escalabilidade.

Probabilidade e estatística

Nível	Critério
Iniciante	Entende média, variância, distribuição e probabilidade básica.
Intermediário	Aplica testes, intervalos de confiança e regressão.
Avançado	Desenha experimentos, bootstrap, Bayes e análise de incerteza.
Sênior	Decide com dados incertos, evita conclusões falsas e orienta produto/engenharia com rigor estatístico.

Cálculo e otimização

Nível	Critério
Iniciante	Entende derivada e função.
Intermediário	Aplica gradiente e gradient descent.
Avançado	Entende backpropagation, Adam, regularização e convexidade.
Sênior	Diagnostica treinamento, escolhe loss, avalia custo e define estratégia de otimização.

Deep Learning e LLMs

Nível	Critério
Iniciante	Entende embeddings, tokens e redes neurais básicas.
Intermediário	Explica softmax, cross-entropy, attention e Transformer.
Avançado	Implementa componentes simplificados e avalia métricas.
Sênior	Decide entre RAG, fine-tuning, LoRA, quantização, modelo aberto/fechado e estratégia de produção.

Retrieval e RAG

Nível	Critério
Iniciante	Entende busca textual, embeddings e similaridade.

Nível	Critério
Intermediário	Implementa busca vetorial e calcula Precision@K/Recall@K.
Avançado	Compara BM25, dense, hybrid, reranking e ANN.
Sênior	Projeta RAG produtivo com avaliação, observabilidade, segurança, custo e melhoria contínua.

5. Aplicação prática em sistemas reais

APIs

A matemática aparece em APIs quando:

- uma endpoint precisa ranquear resultados; uma busca precisa filtrar e ordenar; uma recomendação precisa calcular similaridade; uma regra precisa validar predicados; uma resposta precisa incluir score de confiança.

Exemplo real:

GET /search?q=contrato cancelamento

A API pode usar BM25, dense retrieval, reranking e retornar resultados com score.

Banco de dados

Aplicações:

- índices; cardinalidade; joins; estatísticas do otimizador; hashing; particionamento; busca vetorial; agregações.

Decisão sênior:

- quando usar índice relacional; quando usar search engine; quando usar vector database; quando desnormalizar; quando materializar resultado.

Microsserviços

Aplicações:

- grafos de dependência; análise de impacto; roteamento; tolerância a falhas; topologia; filas e eventos; modelagem de estados.

Decisão sênior:

- identificar acoplamento excessivo; prever efeito cascata; definir comunicação síncrona ou assíncrona; calcular custo de consistência eventual.

Mensageria

Aplicações:

- throughput; latência; ordenação; particionamento; probabilidade de falha; backpressure; retry; deduplicação.

Decisão sênior:

- escolher chave de partição; estimar volume; definir DLQ; controlar idempotência; medir lag.

Segurança

Aplicações:

- lógica de autorização; predicados de acesso; análise de risco; detecção de anomalia; segurança em RAG; prompt injection; vazamento de contexto.

Decisão sênior:

- separar autenticação de autorização; proteger dados sensíveis; restringir retrieval por permissão; validar fonte do contexto; auditar respostas de LLM.

Observabilidade

Aplicações:

- percentis; distribuição de latência; outliers; métricas de erro; correlação; séries temporais; drift; qualidade de resposta.

Decisão sênior:

- monitorar P95/P99; separar erro de modelo, retrieval e infraestrutura; criar alertas úteis; evitar dashboards enganosos.

Testes

Aplicações:

- teste de propriedade; casos extremos; análise combinatória; avaliação estatística; benchmark; teste A/B; golden dataset.

Decisão sênior:

- definir critério de aceitação matemático; medir regressão de qualidade; criar testes para ranking; versionar datasets de avaliação.

Deploy

Aplicações:

- análise de capacidade; custo computacional; latência; memória; GPU/CPU; batch size; cache; escalabilidade horizontal.

Decisão sênior:

- decidir entre batch e online; usar cache de embeddings; comprimir modelo; aplicar quantização; limitar contexto; controlar custo por requisição.

Arquitetura

Aplicações:

- modelagem formal; trade-off; seleção de algoritmo; avaliação de custo; desenho de pipeline; estratégia de evolução.

Decisão sênior:

- escrever ADR; defender arquitetura; apresentar riscos; propor rollback; definir métrica de sucesso.

Performance

Aplicações:

- complexidade algorítmica; vetorização; índice; paralelismo; cache; ANN; compressão; redução de dimensionalidade.

Decisão sênior:

- melhorar gargalo real, não o imaginado; medir antes de otimizar; definir SLO; aceitar aproximação quando necessário; balancear qualidade e latência.

6. Profundidade sênior: trade-offs e decisões

Trade-offs principais

Decisão	Trade-off
BM25 vs dense retrieval	BM25 é bom para termos exatos; dense captura semântica, mas custa mais e pode errar termos específicos.
Busca exata vs ANN	Exata tem maior precisão; ANN reduz latência em escala.
RAG vs fine-tuning	RAG atualiza conhecimento com facilidade; fine-tuning altera comportamento, mas custa mais.
Embedding grande vs pequeno	Grande pode melhorar qualidade; pequeno reduz custo e latência.
Chunk grande vs pequeno	Grande preserva contexto; pequeno melhora precisão de retrieval.
Reranking vs sem reranking	Reranking melhora qualidade; aumenta latência e custo.
Modelo aberto vs fechado	Aberto dá controle; fechado reduz operação, mas aumenta dependência externa.
Quantização vs precisão	Quantização reduz custo; pode degradar qualidade.
Métrica offline vs online	Offline é controlável; online mede impacto real, mas é mais caro e arriscado.

Antipadrões

Usar LLM sem métrica. Criar RAG sem avaliação de retrieval. Escolher algoritmo por moda. Otimizar antes de medir. Ignorar custo por token. Não versionar prompt. Não versionar embedding. Não controlar acesso aos documentos. Confiar apenas em resposta visualmente boa. Não separar latência de busca, reranking e geração. Não documentar decisões arquiteturais.

Estratégias de evolução

Começar com baseline simples. Medir qualidade. Introduzir embeddings. Comparar com BM25. Adicionar busca híbrida. Adicionar reranking. Criar dataset de avaliação. Monitorar produção. Coletar feedback. Ajustar chunking e índice. Avaliar custo. Documentar decisões.

7. Plano de execução sugerido

Duração recomendada

Para domínio sólido, a trilha deve ser feita em aproximadamente:

32 semanas 8 a 12 horas por semana Total estimado: 256 a 384 horas

Para um ritmo mais intenso:

24 semanas 12 a 16 horas por semana Total estimado: 288 a 384 horas

Distribuição semanal

Semanas	Foco	Entregável
1-4	Matemática discreta e lógica	Provas, invariantes e modelagem com grafos.
5-10	Algoritmos e complexidade	80 a 100 problemas + implementações clássicas.
11-15	Álgebra linear	Similaridade, PCA, SVD e embeddings.
16-19	Probabilidade e estatística	A/B test, regressão e análise de incerteza.
20-22	Cálculo e otimização	Gradient descent, loss e regularização.
23-26	Deep Learning e Transformers	Mini attention e mini Transformer.
27-29	Retrieval e RAG	Avaliador de retrieval com métricas.
30-31	RL, RLHF e alignment	Resumos e protótipos conceituais.

Semanas	Foco	Entregável
32	Projeto final e apresentação	Plataforma RAG avaliada + documentação técnica.

Prioridade dos assuntos

Prioridade 1 — Base obrigatória

Matemática discreta. Big O. Estruturas de dados. Grafos. Álgebra linear básica. Probabilidade básica. Derivadas e gradiente.

Prioridade 2 — Aplicação prática

Dynamic programming. PCA/SVD. Testes estatísticos. Regressão. Gradient descent. Embeddings. Similaridade.

Prioridade 3 — IA moderna

Attention. Transformers. Cross-entropy. Perplexity. RAG. Retrieval metrics. ANN. Reranking.

Prioridade 4 — Sênior

Leitura de papers. Decisão arquitetural. Benchmark. Observabilidade. Segurança. Custo. Documentação. Liderança técnica.

Rotina semanal sugerida

Segunda: teoria matemática + anotações Terça: exercícios técnicos Quarta: implementação do zero Quinta: aplicação em projeto real Sexta: revisão e flashcards Sábado: problemas de entrevista + paper Domingo: checkpoint, resumo e planejamento

Checkpoints técnicos

Checkpoint	Critério
Semana 4	Consegue provar corretude de algoritmo simples.
Semana 10	Resolve problemas medium com análise de complexidade.
Semana 15	Implementa similaridade vetorial e explica embeddings.
Semana 19	Interpreta A/B test e intervalo de confiança.
Semana 22	Implementa gradient descent e explica regularização.
Semana 26	Explica self-attention e implementa versão simplificada.
Semana 29	Avalia RAG com métricas de retrieval.
Semana 32	Entrega projeto final documentado com decisões técnicas.

8. Projetos recomendados para portfólio

Projeto 1 — Biblioteca de algoritmos e estruturas de dados

Deve conter:

- arrays; listas; pilhas; filas; hash map simples; árvores; grafos; BFS; DFS; Dijkstra; union-find; DP; testes; análise Big O.

Objetivo: mostrar domínio de base computacional.

Projeto 2 — Sistema de busca semântica

Deve conter:

- ingestão de textos; geração de embeddings; cosine similarity; ranking; API REST; testes; avaliação simples.

Objetivo: demonstrar álgebra linear aplicada.

Projeto 3 — Avaliador de experimentos A/B

Deve conter:

- entrada de métricas; cálculo de conversão; intervalo de confiança; teste de hipótese; visualização; recomendação técnica.

Objetivo: demonstrar estatística aplicada a produto.

Projeto 4 — Mini Transformer educacional

Deve conter:

- tokenização simples; embeddings; positional encoding; self-attention; feed-forward; explicação matemática; testes de shape.

Objetivo: demonstrar entendimento de deep learning.

Projeto 5 — RAG evaluator completo

Deve conter:

- documentos; chunking; embeddings; BM25; dense retrieval; hybrid search; reranking; métricas de retrieval; API; logs; relatório técnico.

Objetivo: demonstrar capacidade avançada/sênior em IA aplicada.

9. Checklist de domínio técnico

Fundamentos

- Sei explicar proposições, predicados, conjuntos, relações e funções.
- Sei provar algoritmos simples por indução.
- Sei identificar invariantes.
- Sei modelar problemas como grafos.
- Sei resolver recorrências básicas.

Algoritmos

- Sei calcular Big O, Big Ω e Big Θ .
- Sei implementar sorting/searching.
- Sei usar hash map corretamente.
- Sei resolver problemas com árvores.
- Sei resolver problemas com grafos.
- Sei aplicar DP, greedy e backtracking.
- Sei resolver problemas medium/hard com justificativa.

Álgebra linear

- Sei operar vetores e matrizes.

- Sei explicar produto escalar, norma e distância.
- Sei aplicar cosine similarity.
- Sei explicar embeddings.
- Sei aplicar PCA/SVD em alto nível.
- Sei entender operações com tensores.

Estatística

- Sei explicar distribuições.
- Sei calcular esperança e variância.
- Sei interpretar correlação.
- Sei aplicar Bayes em problemas simples.
- Sei interpretar p-value corretamente.
- Sei desenhar A/B test.
- Sei usar intervalo de confiança.

Otimização

- Sei explicar derivada e gradiente.
- Sei implementar gradient descent.
- Sei explicar backpropagation.
- Sei diferenciar loss e métrica.
- Sei aplicar regularização.
- Sei diagnosticar overfitting.

Deep Learning e LLMs

- Sei explicar softmax.
- Sei explicar cross-entropy.
- Sei explicar attention.
- Sei explicar self-attention.
- Sei explicar Transformer.
- Sei explicar perplexity.
- Sei entender quantization.
- Sei comparar RAG, fine-tuning e prompt engineering.

Retrieval e RAG

- Sei implementar TF-IDF ou BM25 simples.
- Sei usar embeddings para busca.
- Sei calcular Recall@K.
- Sei calcular Precision@K.
- Sei calcular MRR.
- Sei calcular NDCG.
- Sei comparar lexical, dense e hybrid search.
- Sei projetar avaliação de RAG.
- Sei monitorar qualidade em produção.

Papers

- Resumi Attention Is All You Need.
- Resumi BERT.
- Resumi GPT-3.
- Resumi RAG.
- Resumi ReAct.
- Resumi LoRA.
- Resumi QLoRA.
- Resumi ColBERT.
- Resumi DDPM.
- Implementei pelo menos um protótipo baseado em paper.

10. Indicadores de prontidão para nível avançado/sênior

O estudante está pronto para atuar em nível avançado/sênior quando consegue:

- Explicar conceitos matemáticos com aplicação prática. Não apenas definir gradiente, embedding ou complexidade, mas mostrar onde isso impacta um sistema real. Resolver problemas algorítmicos medium/hard. Com análise de complexidade, clareza de código e justificativa. Projetar um sistema RAG completo. Incluindo chunking, embedding, retrieval, reranking, geração, avaliação, segurança e observabilidade. Separar erro de modelo, erro de retrieval e erro de produto. Essa é uma competência essencial em sistemas com LLM. Ler papers e transformar em protótipos. O objetivo não é apenas resumir. É implementar a menor versão útil da ideia. Tomar decisões com trade-offs claros. Por exemplo: BM25 versus dense, RAG versus fine-tuning, exato versus aproximado, qualidade versus custo. Documentar decisões técnicas. Com ADRs, RFCs, benchmarks e métricas. Avaliar impacto em produção. Incluindo latência, custo, segurança, escalabilidade, manutenção e risco. Orientar outros desenvolvedores. Explicando fundamentos, revisando soluções e elevando o padrão técnico do time.

Resumo final da trilha

A Trilha Matemática Aplicada forma a base necessária para sair do uso superficial de algoritmos e IA e alcançar domínio técnico real.

Ela começa com lógica, provas e matemática discreta; avança para algoritmos e complexidade; consolida álgebra linear, estatística e otimização; aplica esses fundamentos em deep learning, Transformers, LLMs, retrieval e RAG; e finaliza com papers, RLHF, decisão arquitetural e liderança técnica.

Ao final, o profissional deve ser capaz de:

- resolver problemas algorítmicos complexos; entender matemática de modelos modernos; avaliar sistemas de IA com métricas corretas; construir RAG com qualidade mensurável; ler papers relevantes; justificar decisões técnicas; projetar soluções escaláveis, seguras e mantíveis; atuar com postura de engenheiro avançado/sênior em software e IA.